基于约束编程的分布保持测试采样方法

作者姓名

所属机构，所在城市，邮政编码，国家

通信作者：作者姓名，电子邮箱：author@example.com

短标题：约束编程测试采样

摘要

复杂测试输入往往受到合法性条件、跨字段关系和测试目标等约束。若先随机生成测试数据再过滤，严格约束会造成大量无效候选；若直接采用约束规划求解器返回的可行解，求解器的搜索次序又无法表达测试输入的预期分布。本文研究一种基于约束编程的测试采样方法：首先从给定先验分布中独立产生候选输入，再由约束规划求解器判断候选是否可补全，并为可行候选构造见证。由此得到的测试输入精确服从先验分布在可行集合上的条件分布。

为减少不可行候选引起的重复求解，本文进一步从失败查询中提取并泛化不可行区域，以该区域在先验分布下新增覆盖的概率质量衡量其价值，并在有限的额外求解预算内安排区域搜索。经过证明的不可行区域只会提前拒绝原本必然失败的候选，因此不会改变输出分布。本文给出约束测试采样的形式化模型、分布保持条件以及概率质量引导的在线采样框架。

关键词：约束编程；软件测试；测试生成；条件采样；不可满足核；概率质量

1 引言

测试生成面对的困难，常常不是找不到输入，而是绝大多数容易生成的输入都没有意义。在配置验证、资源分配和复杂业务规则等场景中，一个测试是否成立，通常取决于输入合法性、跨字段关系、资源限制和测试目标等多组彼此关联的条件。随着约束数量增加，可行测试在全部组合中所占的比例可能迅速下降。

最直接的方法是随机产生候选并丢弃不满足约束的输入。这种拒绝采样概念简单，也能保留原始分布，但会反复检查相似的不可行组合。当可行区域稀疏时，大部分计算都消耗在注定失败的候选上。另一种方法是把全部条件交给约束规划求解器，由求解器直接构造一个可行解。这能够避免无效输入，却引入了另一项问题：求解器寻找解的顺序服务于搜索效率，并不代表测试场景中输入出现的自然频率。先找到的解不等于应当被频繁测试的解，重复调用求解器也不自然地产生一个可解释的测试分布。

这形成了本文关注的核心矛盾：测试生成既需要约束规划提供可行性保证，又需要概率采样保留输入分布。前者回答“哪些测试可能成立”，后者回答“不同测试应以怎样的相对频率出现”。若将二者简单合并，往往会牺牲其中一个目标。本文尝试给出一种明确的分工：概率模型负责提出候选输入，约束规划负责判定候选能否补全，并为其构造满足约束的见证。

然而，仅仅把求解器放在采样过程之后仍不够。当大量候选因相同或相近的原因失败时，框架会对同一片不可行区域反复求解。一次失败查询实际上包含了可复用的信息：失败可能只由候选输入中的少数绑定共同造成，其他取值与冲突无关。若能安全地识别并缓存这样的不可行区域，后续落入该区域的候选便可直接拒绝。关键问题随之从“能否缓存失败”转变为“在有限计算预算下，哪些失败区域最值得被泛化和缓存”。

本文认为，答案不应只取决于区域由多少个条件描述。对于偏斜或相关的输入分布，一个描述更简短的区域未必更常出现；一个看似较大的区域也可能与已有缓存高度重叠。真正与后续测试成本直接相关的，是该区域在当前缓存之外新增覆盖了多少先验概率质量。基于这一观察，本文把分布保持的条件采样与概率质量引导的不可行区域学习结合起来，构成一个面向约束系统的测试采样框架。

设测试输入为随机变量 X，其先验分布为 π；其余需要由约束规划补全的变量为 Y；完整测试必须满足约束 $R(X,Y)$。本文希望生成一列测试

$$(X^{(1)},Y^{(1)}),\ldots,(X^{(n)},Y^{(n)}),$$

使输入 $X^{(j)}$ 服从 π 在可补全集合上的条件分布，而 $Y^{(j)}$ 是相应的可行见证。与此同时，算法应复用失败查询所揭示的不可行区域，在不改变输出分布的前提下减少后续约束求解。

这里必须区分“使用算法”和“提出算法”两个层次。约束规划求解器内部当然运行搜索、传播和回溯算法，但本文把它视为一个提供可满足性判定与见证构造能力的基础组件。单次调用求解器不是本文的算法贡献。本文所研究的算法，是如何组织候选采样、可行性查询、冲突区域泛化、概率质量评价、缓存更新与预算调度，从而获得可证明的分布语义和可测量的查询收益。

基于上述设定，本文集中回答三个相互关联的问题：（1）如何在约束系统上定义具有明确概率语义的测试输入，使直接采样的变量服从目标分布，而求解得到的变量仅承担见证作用；（2）如何从失败查询中学习可复用的不可行区域，并保证这种学习不会改变最终的输入分布；（3）如何在有限预算下，根据区域新增覆盖的概率质量安排泛化与缓存，从而减少重复的约束规划调用。统一的约束采样模型为这些问题提供形式化基础，算法效果则通过后续实验问题进行评价。

图 1 展示本文方法的基本流程。每轮首先按先验分布产生候选输入。若候选命中已经证明的不可行区域，则直接拒绝；否则调用约束规划求解器。SAT 候选与求解见证共同形成测试样本；UNSAT 候选立即拒绝，其原始安全区域可以直接进入缓存，并

产生可继续泛化的学习任务。额外泛化按预算与采样交错执行，不是拒绝当前候选的前提。未决或执行失败的查询返回诊断，不作为不可行证据。

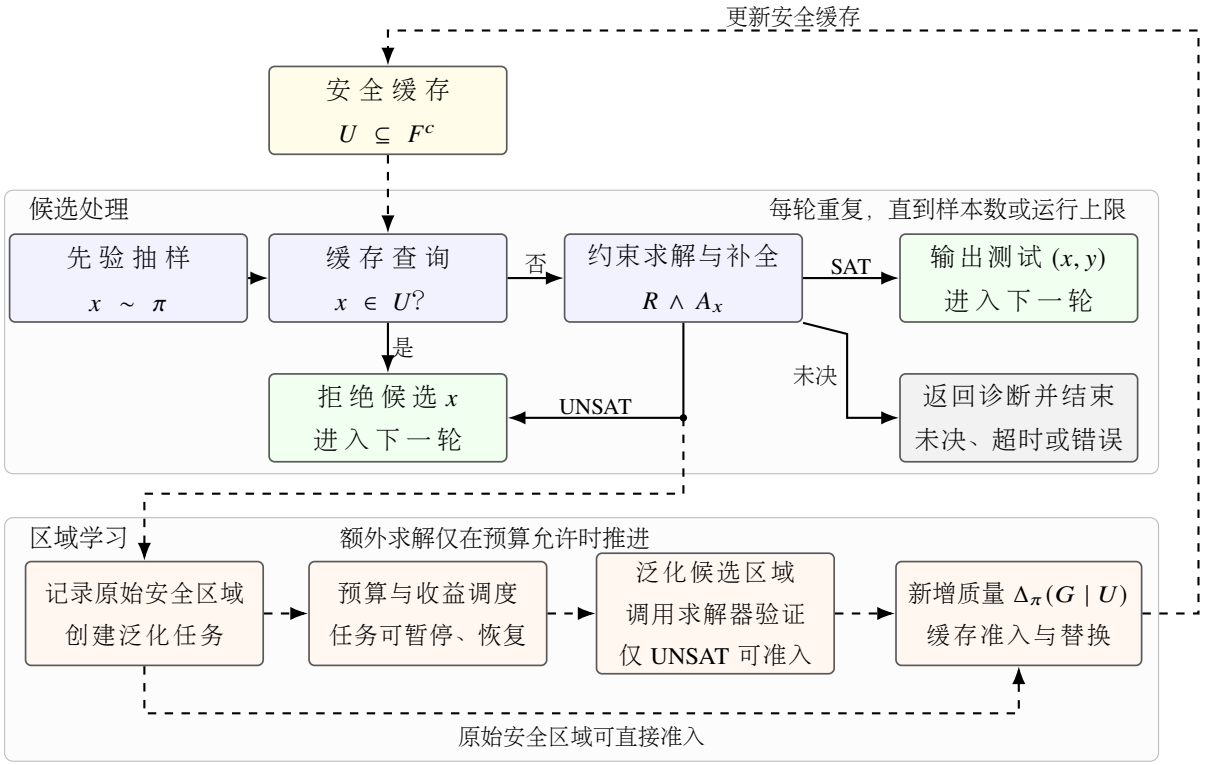


图 1: 分布保持测试采样与安全区域学习。实线表示候选处理，虚线表示任务、区域及缓存信息流，不表示并行执行。UNSAT 候选立即拒绝，原始安全区域可直接准入；额外泛化按预算交错推进，只有经 UNSAT 证明的区域才能用于缓存拒绝。未决结果不作为不可行证据。

图中将候选处理与区域学习分开，并显式画出二者共享的安全缓存 U。上方流程决定候选是否成为测试样本；下方流程在预算内发现和选择可以安全跳过的不可行区域。暂停额外学习不会阻止候选处理继续执行；缓存更新只改变后续查询成本，不改变可行输入的接受集合。

本文作出以下三项贡献。

1. 建立一个面向约束系统的测试采样模型，以统一约束关系描述测试输入、求解补全变量和测试目标，并明确概率变量与见证变量的边界。
2. 定义一种分布保持的测试采样语义与基本过程：从先验分布产生候选，以约束规划完成可行性判定和见证补全，并保证输出输入服从可行集合上的条件分布。
3. 提出概率质量引导的不可行区域学习方法，以相对现有缓存的新增覆盖质量衡量区域价值，在有限额外查询预算内进行在线泛化和调度，并分析其安全性、分布保持性质及期望查询收益。

其中，第一项是问题表示基础，第二项建立统计正确性，第三项构成主要算法贡献。本文不把约束规划的一般求解能力、某种特定建模形式或求解器内部算法作为创

新主张。

本文其余部分组织如下。第 2 节讨论相关工作。第 3 节定义约束测试采样问题及其目标分布。第 4 节给出概率质量引导的测试采样方法，第 5 节分析其理论性质，第 6 节进行实验评价。第 7 节讨论适用边界、扩展方向和有效性威胁，第 8 节总结全文。

2 相关工作

3 问题定义与数学模型

3.1 基本符号与假设

本文讨论有限但可能很大的离散测试空间。对任意有限集合 A，记 $|A|$ 为其元素个数；对命题 P，记 $\mathbf{1}[P]$ 为其指示函数。概率均相对于明确给定的输入先验分布定义。约束规划求解器提供如下抽象能力：给定约束及一组变量绑定，返回可行、不可行或未决；当结果为可行时可以返回一个满足约束的见证，当结果为不可行时可以返回一组足以导致冲突的绑定。本文的正确性结论只使用可靠的可行或不可行结果，未决结果不用于接受样本或学习不可行区域。

3.2 约束测试采样模型

3.2.1 概率变量与补全变量

为了给测试生成赋予清晰的概率语义，本文将参与测试生成的变量划分为两组。记

$$X = (X_1, \ldots, X_d) \in \mathcal{X}$$

为直接按照用户给定分布采样的测试输入，记

$$Y = (Y_1, \ldots, Y_e) \in \mathcal{Y}$$

为由约束规划求解器补全的变量。X 决定测试输入的概率语义；Y 用于补充使测试满足全部约束所需的信息，但不预设概率分布。二者的划分由测试目标决定：需要控制出现频率的变量属于 X，只需存在一个合法取值的辅助变量属于 Y。

将输入合法性、跨字段关系、资源限制和测试目标统一表示为约束

$$R(X, Y).$$

其中，$R(x, y)$ 为真表示完整赋值 (x, y) 构成一个符合测试条件的样本。因此，测试生成被归结为：先按照给定分布选择 x，再判断是否存在 y 使 $R(x, y)$ 成立。

3.2.2 可行输入与见证

定义可行输入集合

$$F = \{x \in \mathcal{X} \mid \exists y \in \mathcal{Y} : R(x, y)\},$$

不可行输入集合为 $F^c = \mathcal{X} \setminus F$。对任意 $x \in F$，其见证集合为

$$W(x) = \{y \in \mathcal{Y} \mid R(x, y)\}.$$

这一定义明确了求解器的职责。它判断 x 是否属于 F，并在 $x \in F$ 时返回任意 $y \in W(x)$。除非另行定义见证分布，否则求解器返回不同见证的频率不属于本文的概率保证。

3.3 测试采样目标

3.3.1 业务先验与条件分布

设测试输入 X 具有联合先验概率质量函数

$$\pi(x) = \Pr[X = x], \qquad \sum_{x \in \mathcal{X}} \pi(x) = 1.$$

π 描述施加可行性约束之前不同测试输入的相对重要性或自然出现频率，允许变量之间存在相关性。记可行概率为

$$Z = \Pr_{\pi}[X \in F] = \sum_{x \in F} \pi(x).$$

当 $Z > 0$ 时，本文的目标输入分布为

$$\mu(x) = \Pr_{\pi}[X = x \mid X \in F] = \frac{\pi(x)\mathbf{1}[x \in F]}{Z}.$$

对任意 $x, x' \in F$，有

$$\frac{\mu(x)}{\mu(x')} = \frac{\pi(x)}{\pi(x')}.$$

因此，条件化只移除不可行输入，不改变可行输入之间原有的相对概率。只有当先验在 F 上均匀时，μ 才是可行输入上的均匀分布。

3.3.2 输出语义

若算法输出 n 个测试 $(X^{(j)}, Y^{(j)})$，本文要求

$$X^{(1)}, \ldots, X^{(n)} \overset{\text{iid}}{\sim} \mu, \qquad Y^{(j)} \in W(X^{(j)}), \quad j = 1, \ldots, n.$$

输入允许重复，因为重复是概率分布的自然结果。概率保证只作用于 X；Y 是使测试完整的约束见证。若某项补全变量也需要满足特定频率，应将其纳入 X，或显式增加相应的条件概率模型。

基本拒绝采样能够达到这一目标：反复从 π 独立采样 x，仅当 $x \in F$ 时接受。本文在此基础上加入安全缓存，以降低判断 $x \in F$ 所需的约束规划调用次数。

3.4 不可行区域与安全缓存

3.4.1 冲突绑定表示的不可行区域

对候选输入 $x = (x_1, \ldots, x_d)$，定义完整绑定集合

$$A_x = \{X_1 = x_1, \ldots, X_d = x_d\}.$$

若约束规划求解器证明 $R \wedge A_x$ 不可行，并找到子集 $C \subseteq A_x$，满足

$$R \wedge C \text{ 不可行},$$

则 C 表示一个不可行区域

$$G(C) = \{x' \in \mathcal{X} \mid x' \models C\}.$$

未出现在 C 中的输入变量可以自由取值。由 $R \wedge C$ 不可行可知

$$G(C) \cap F = \varnothing.$$

因此，任何落入 $G(C)$ 的候选都不可能被补全为有效测试。

设安全缓存包含若干已经证明的不可行区域，其并集为

$$U = \bigcup_{C \in \mathcal{C}} G(C).$$

缓存必须始终满足不变量 $U \subseteq F^c$。算法可直接拒绝所有 $x \in U$，而无需再次调用求解器。第 5 节将证明，只要该不变量始终成立，缓存就不会改变输出输入的条件分布。

3.5 概率质量引导的区域泛化目标

3.5.1 新增覆盖概率质量

对任意输入区域 $G \subseteq \mathcal{X}$，定义其先验概率质量为

$$\pi(G) = \Pr_{\pi}[X \in G].$$

当缓存已经覆盖区域 U 时，新区域 G 的实际增量不是 $\pi(G)$，而是

$$\Delta_\pi(G \mid U) = \Pr_{\pi}[X \in G \setminus U] = \pi(U \cup G) - \pi(U).$$

对冲突集合 C，简记

$$\Delta_\pi(C \mid U) = \Delta_\pi(G(C) \mid U).$$

该量表示一个新的独立候选因加入区域 $G(C)$ 而额外命中缓存的概率，也就是该区域每轮能够避免一次重复可行性查询的概率。

不考虑区域搜索成本时，质量最优冲突集合定义为

$$C^* \in \arg\max_{C \subseteq A_x} \Delta_\pi(C \mid U) \quad \text{s.t.} \quad R \wedge C \text{ 不可行}.$$

这一目标不同于单纯追求最少绑定。先验偏斜、变量相关以及区域重叠都会使绑定数量相同的区域具有显著不同的实际价值。

3.5.2 考虑额外查询成本的目标

设从当前时刻起预计还会产生 H 个候选输入，从初始冲突得到候选区域 $G(C)$ 需要增加 $r(C)$ 次约束规划调用。以调用次数衡量，该区域的期望净收益为

$$J_H(C \mid U) = H\,\Delta_\pi(C \mid U) - r(C).$$

第一项是未来缓存命中的期望次数，第二项是当前为泛化区域支付的额外查询成本。相应的预算化目标为

$$C_H^* \in \arg\max_{C \subseteq A_x} J_H(C \mid U) \quad \text{s.t.} \quad R \wedge C \text{ 不可行}.$$

调用次数是一项与具体求解器相对解耦的机制指标，但不等同于总运行时间。实验还需分别记录单次求解耗时、区域质量计算成本、缓存匹配成本和端到端时间。

4 概率质量引导的测试采样方法

4.1 分布保持的基本采样过程

4.2 概率质量引导的不可行区域泛化

4.3 在线采样与预算调度

设算法已经处理 t 个原始候选，累计用于区域泛化的额外约束规划调用数为 $r(t)$。本文采用随采样过程缓慢增长的预算

$$B(t) = \lfloor \beta \sqrt{t} \rfloor, \qquad \beta > 0,$$

并要求

$$r(t) \le B(t).$$

该预算允许算法持续吸收新的失败信息，同时保证额外调用相对于候选总数的比例趋于零。每轮主采样结束后，算法在剩余预算内推进尚未完成的区域泛化任务，优先处理新增覆盖质量上界较高的任务，并以周期性公平调度避免长期搁置低优先级任务。

在这一模型下，一次约束规划调用只是算法可计量的基本操作。算法研究的对象是调用之间的决策结构：何时调用、对哪些绑定调用、如何利用返回的冲突信息、何时停止泛化，以及如何用新知识影响后续采样。这样的界定既承认求解器算法的重要性，也避免把通用求解能力误作本文的新贡献。

4.4 算法复杂度

5 理论分析

5.1 安全性与分布保持性

命题 **1**（安全缓存不改变输出分布）。 若候选序列独立同分布地服从 π，缓存始终满足 $U \subseteq F^c$，且所有未命中缓存的候选均由可靠的可行性判定决定接受或拒绝，则算法接受的输入序列独立同分布地服从 μ。

证明。 缓存只提前拒绝 F^c 中的候选，不会拒绝任何 F 中的候选。因而，对同一条候选序列，带缓存算法与基本拒绝采样接受完全相同的子序列。基本拒绝采样的接受序列服从 π 在事件 $X \in F$ 下的条件分布 μ，故结论成立。 □

这一性质给出了区域学习的安全边界：算法可以改变不可行候选的判定成本，但不能以启发式方式跳过未经证明的区域，否则可能改变接受子序列并引入采样偏差。

5.2 区域泛化的安全性与近似性质

5.3 预算、收敛与查询收益

6 实验评价

6.1 研究问题

6.2 实验对象与数据生成

6.3 比较方法

6.4 评价指标与统计方法

6.5 实验结果

7 讨论

7.1 适用范围与方法边界

本文聚焦于有限测试空间中的测试生成问题，并假设约束规划过程能够对研究范围内的问题给出可靠的可行或不可行结论。测试输入的先验分布可以是非均匀且相关的；求解器补全的见证暂不具有概率语义。超出这一范围的结论需要额外假设或近似分析。

7.2 扩展方向

外部黑盒系统是第一个扩展方向。将生成的测试输入提交给外部被测系统，并根据真实输出检查测试性质，可把本文方法扩展为生成、执行与判定组成的闭环。该扩展需要进一步处理输出观测、环境控制和执行失败，但不改变本文对输入采样的基本定义。

具有内部随机性的被测系统是第二个扩展方向。若系统在相同输入下可能产生不同结果，则需要显式区分输入分布、系统响应分布和求解器见证选择。相应研究可进一步考察随机响应下的覆盖、统计判定和重复试验分配。

7.3 有效性威胁

8 结论